

Overview:

This project involves formally specifying a protocol descriptor for a multi-client email system, then implementing that protocol in Java using TCP as the transmission encoding. The server follows a **thread-per-client** model, where each client connection is handled in a **separate thread**. The server maintains **session state**, allowing users to log in, send, receive, and store emails. All state must be stored and manipulated in memory - no database storage may be used. You may choose to load information from a database/file on server start-up and store information to a database/file on server shutdown, but these may not be accessed during the run of the server.

Key Features:

- Authentication: Users can log in via a client and send emails to other registered users.
- Data storage: The server stores emails in-memory and allows users to access to their sent and received emails.

Overall Submission Requirements:

- 1) A complete protocol descriptor (as seen in class) for your email system protocol
- 2) A TCP-based thread-per-client server providing this protocol
- 3) A well-designed TCP-based client.

Protocol Descriptor requirements:

Your protocol descriptor should be a pdf document included as part of your git repository. It should feature the following:

- 1) Service location information.
- 2) Appropriate data encoding.
Note: Allowable data encoding options include text, object or json
- 3) Transmission encoding (this must be stated, even though this is explicitly to be a TCP-based system)
- 4) Request structure and all possible responses (plus their formats) for each protocol command
- 5) Sample exchanges for every request (you must include a sample for each possible response)

Functional Requirements:

Note: All communication between client and server should adhere to your protocol descriptor. Marks will be loss if the protocol descriptor details do not match the implementation.

Server Functionality:

- 1) Accept multiple clients using a thread-per-client model.
- 2) Store user credentials
- 3) Store emails
- 4) Authenticate users
- 5) Manage client sessions:
 - a. Track login state and username.
- 6) Handle email operations:
 - a. Send an email.
 - b. Retrieve received email metadata (ID, sender, subject, timestamp).
 - c. Search received emails and send back matching metadata (ID, sender, subject, timestamp)
 - d. Retrieve sent email metadata (ID, recipient, subject, timestamp).
 - e. Search sent emails and send back matching metadata (ID, recipient, subject, timestamp)
 - f. Retrieve the content of a specific email.
 - g. Logout/end session.
- 7) Handle client disconnections gracefully to ensure the server remains functional.

Client Functionality:

- 1) Connect to the server using a pre-defined hostname and port.
- 2) Login/Register using a username and password.
- 3) Send emails to other registered users.
- 4) List received emails (only metadata (id, sender, subject, timestamp). No content should be shown).
- 5) Search received emails (only metadata should be shown)
- 6) List sent emails (only metadata, no content)
- 7) Search sent emails (only metadata should be shown)
- 8) Read specific emails based on email ID.
- 9) Logout to end the session.
- 10) Exit to end the program.

Non-Functional Requirements:

- 1) Security should be appropriately considered (e.g. hashing of passwords to protect data when stored, using secure sockets to protect data during transmission)
- 2) Proper exception handling to avoid crashes due to unexpected disconnections (server and client)
- 3) Implement **logging** for debugging and tracking user activities.
- 4) Layered software design including use of interfaces.
- 5) Appropriate user experience considerations for client-side view.

Development Approach:

This CA is broken into three distinct sections:

- | | |
|--|-----|
| 1) Protocol descriptor | 10% |
| 2) TCP server using a thread-per-client design | 50% |
| 3) TCP client | 40% |

Section 1 (the protocol descriptor) should be developed before attempting sections 2 and 3 (server and client respectively) as the protocol dictates what information will be exchanged between the client and server. Sections 2 and 3 can be completed in parallel or one after the other, but in the case of developing them one after the other, it's advisable to complete the server before attempting the client, as you can easily test the server logic with a "clear" client.

Notes and Hints to Remember:

- **Test your code as you go!** Do not assume your model layer classes work without testing them, write proper unit tests to check they are working
- Consider providing "**bootstrapping**" methods – methods that will populate your model with some dummy information for you to test against. These may pull from a file or database on server start-up. This will avoid you needing to create emails and user information through user interaction BEFORE you can do anything with the system.

Screencast Details:

You are required to provide a screencast demonstrating your system in action and explaining its functionality. In this screencast, each team member should outline the overall system design, followed by demonstrating and explaining their own contributions to the system. Where functionality was developed by both team members, each member should outline the way the functionality works overall, then should explain what they contributed in more detail.

Submission instructions:

Upload a link to a github repository containing your project, plus a zipped folder containing your screencasts or a link to where these files can be accessed. Only one upload is required per group.

Deadline: Tuesday 5th May 23:59